

# Workshop: CI/CD for Azure Container Apps (60 min)

Hands-on walkthrough of this repo's Azure DevOps pipelines: infrastructure as code with Bicep, container CI, and revision-based blue/green deployment on Azure Container Apps (ACA).

## Preparation checklist (before attendees arrive)

- [ ] Azure DevOps project with the repo imported; service connection created.
- [ ] Pipeline variables set: `azureServiceConnection`, `acrName`, `resourceGroupName` ([azure-pipelines.yml](#)), `environmentName`, `location` ([azure-pipelines-infra.yml](#)).
- [ ] Infra provisioned and both pipelines green at least once.
- [ ] **~20 min before start:** push a small UI change to `main` so a run is already paused at the `CanaryStore` gate — build + push takes too long to run live end-to-end.
- [ ] Tabs open: Azure portal (resource group + store app → *Revisions and replicas*), the store URL, the Azure DevOps run.

**Fallback plan:** if the live run misbehaves, drive traffic manually:

```
az containerapp ingress traffic set -n <app> -g <rg> --revision-weight <rev1>=50 <rev2>=50
```

## Agenda

| Time  | Segment                | Content / demo beats                                                                                                                                                                                                                                                      |
|-------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0–5   | Welcome & goals        | What "good CI/CD for containers" looks like: repeatable infra, tested images, progressive delivery, instant rollback.                                                                                                                                                     |
| 5–13  | Architecture tour      | Repo walk-through: 3 services (Blazor Server store + 2 minimal APIs), Dockerfiles, <a href="#">infra/main.bicep</a> (subscription-scope, <code>azd-service-name</code> tags), topology diagram in the <a href="#">README</a> .                                            |
| 13–21 | Infra pipeline (IaC)   | <a href="#">azure-pipelines-infra.yml</a> : PRs run <code>validate</code> + <code>what-if</code> only, pushes to main deploy; idempotent re-provisioning ( <code>*AppExists</code> pattern preserves running images). Show a what-if log.                                 |
| 21–29 | CI: build → test → ACR | <a href="#">azure-pipelines.yml</a> stages <code>Build</code> / <code>PublishImages</code> : test gating, matrix docker builds, <code>\$(Build.BuildId)</code> tagging (never rely on <code>:latest</code> ).                                                             |
| 29–35 | ACA revisions 101      | Single vs multiple revision mode; a revision = immutable snapshot of the app template; traffic weights; revision-scoped vs app-scoped settings; why the store is <code>Multiple</code> in <a href="#">infra/app/store.bicep</a> while the APIs stay <code>Single</code> . |

| Time  | Segment                      | Content / demo beats                                                                                                                                                                                                                      |
|-------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 35–50 | <b>Live demo: blue/green</b> | See demo script below.                                                                                                                                                                                                                    |
| 50–55 | Production considerations    | Health probes, min replicas per active revision (cost), session affinity, DB schema compatibility across revisions, cleaning up old revisions, ADO Environments + checks as the enterprise alternative to <code>ManualValidation</code> . |
| 55–60 | Q&A + resources              | <a href="#">ACA revisions</a> , <a href="#">ACA + Azure Pipelines</a> , this repo.                                                                                                                                                        |

## Demo script (35–50 min)

1. Show the commit that changed the UI — the version badge in `MainLayout.razor` renders `CONTAINER_APP_REVISION`, so the serving revision is visible in the page header.
2. Walk the paused pipeline run: `DeployStoreGreen` created the green revision at **0%** traffic; open the revision-specific preview URL from the logs.
3. Show both revisions and weights in the portal (*Revisions and replicas*).
4. Approve the `CanaryStore` gate → **90/10**. Refresh the store URL a few times; the badge occasionally flips to green.
5. Approve the `ShiftStore` gate → **50/50**. Show updated weights in the portal.
6. **Reject the `PromoteStore` gate** → the `RollbackStore` stage restores 100% to blue and deactivates green within seconds.
7. (Optional) Re-run and approve all gates to promote green to 100%.

## Expected questions & answers

1. **Why is only the store in multiple-revision mode?** Blue/green adds value where users see change (external ingress). The APIs are internal and consumed via service name; single-revision direct update keeps them simple. The same pattern can be applied to them if needed.
2. **Why pin traffic to blue before deploying?** In multiple-revision mode the default traffic rule is `latestRevision=true` (100% to newest). Pinning 100% to the current revision first ensures the new revision starts at 0%.
3. **What exactly is a revision — is it a replica?** No. A revision is an immutable snapshot of the app template (image, env vars, scale rules). Each active revision scales its own replicas independently.
4. **Which changes create a new revision?** Revision-scoped changes: image, env vars, scale rules, resources. App-scoped changes (ingress config, secret values, traffic weights) do not create revisions.

5. **How is rollback "instant"?** The old revision stays provisioned and warm at 0% weight; rollback is just a traffic-weight update — no image pull, no cold start. That is also why the pipeline does not deactivate blue at promote time.
6. **Does a 90/10 split affect in-flight requests or sessions?** Weights apply to new connections. Blazor Server holds a SignalR/WebSocket circuit, so an open session stays on one revision; a refresh re-rolls the dice. Enable session affinity if users must stay pinned.
7. **What does running two revisions cost?** Each active revision runs at least its `minReplicas`. Here that means 1+1 while a rollout is in flight — deactivate old revisions when done to stop paying for them.
8. **Why `ManualValidation` instead of Environment approvals?** It keeps everything in one YAML file with zero org setup — ideal for a demo/workshop. In production, prefer deployment jobs targeting ADO Environments with approvals, business-hours checks, and audit history.
9. **How does the app know which revision it is?** ACA injects `CONTAINER_APP_REVISION` (plus `CONTAINER_APP_NAME`, `CONTAINER_APP_HOSTNAME`) into every replica; the badge just reads the env var.
10. **How do I test green before it gets any traffic?** Every revision gets its own FQDN ( `<app>--<suffix>.<region>.azurecontainerapps.io` ); the pipeline prints it. Revision *labels* can provide stable preview URLs.
11. **What about database/schema changes?** Blue and green run simultaneously, so schema changes must be backward-compatible (expand/contract pattern). Blue/green doesn't solve data migrations — it exposes them.
12. **Could two overlapping runs conflict?** Yes — the pattern assumes one rollout in flight. Each run uses a unique `--revision-suffix b<buildId>`, but a second run would re-pin traffic. Guard with an exclusive lock on an ADO Environment or batched triggers.
13. **Is this the same as App Service deployment slots?** Conceptually similar (staged versions + traffic control), but revisions are immutable, you can keep many, and traffic splitting is percentage-based across several revisions — not just a slot swap.
14. **Why look up apps by the `azd-service-name` tag?** Resource names are generated ( `resourceToken` ); the tag decouples the pipeline from naming so `azd` and Azure DevOps can coexist.
15. **Can I do canary automatically instead of manual gates?** Yes — replace the gates with a job that watches Application Insights (failed-request rate for the green revision) and promotes or rolls back based on thresholds; the traffic commands stay identical.
16. **Does re-running the infra pipeline break an in-flight rollout?** It can — the Bicep template doesn't manage traffic weights, and a template redeploy resets traffic to latest. Finish or roll back before re-provisioning (noted in the README).